Tailwind CSS Tutorial: Beginner Level

For people who know HTML and CSS fundamentals (selectors, box model, Flexbox, Grid) and want to learn Tailwind.

Table of Contents

1. What Tailwind Actually Is
2. Getting It Running
3. The Core Idea: Utility Classes
4. Spacing
5. Colors
6. Typography
7. Layout: Flexbox and Grid Utilities
8. Sizing and Borders
9. States: Hover, Focus, and More
10. Responsive Design
11. Practice Project: A Profile Card
12. Where to Go Next

---

1. What Tailwind Actually Is

Regular CSS asks you to invent class names and then write the styles separately:

```
.card { padding: 16px; border-radius: 8px;
background: white; }
```

Tailwind flips this around. Instead of writing custom CSS, you apply small, single-purpose classes directly in your HTML, and Tailwind's build process only generates the CSS you actually used:

```
<div class="p-4 rounded-lg bg-white">
```

Each class does one job — `p-4` is padding, `rounded-lg` is a border radius, `bg-white` is a background color. You compose them together instead of switching between HTML and a CSS file.

The trade-off worth knowing upfront: your HTML gets more classes, but you stop naming things and stop context-switching files. Most people find this awkward for the first few hours and then much faster afterward.

---

2. Getting It Running

The simplest way to try Tailwind without any build tooling is the CDN script, good for learning and quick prototypes (not recommended for production):

```
<head>
  <script src="https://cdn.tailwindcss.com">
```

```
</script>
</head>
```

For a real project, Tailwind is installed via npm and integrated into your build tool:

```
npm install -D tailwindcss
npx tailwindcss init
```

This generates a `tailwind.config.js` file where you'll later customize colors, spacing, and fonts. For now, the CDN version is the easiest way to follow along with this tutorial in a single HTML file.

---

3. The Core Idea: Utility Classes

Nearly every Tailwind class name follows a pattern: `property-value`.

```
<div class="bg-blue-500 text-white p-4 rounded-
md shadow-lg">
  Hello Tailwind
</div>
```

Breaking that down:

- `bg-blue-500` — background color, blue, shade 500
- `text-white` — text color, white
- `p-4` — padding on all sides
- `rounded-md` — medium border radius

- `shadow-lg` — large drop shadow

There's no CSS file to open — everything about this element's appearance is visible directly in the HTML.

---

4. Spacing

Tailwind's spacing scale is a set of consistent steps (not raw pixels), which keeps designs visually consistent:

```
<div class="p-4">Padding on all sides</div>
<div class="px-4">Padding left and right</div>
<div class="py-4">Padding top and bottom</div>
<div class="pt-4">Padding top only</div>

<div class="m-4">Margin on all sides</div>
<div class="mx-auto">Auto left/right margin
(centers a block with a set width)</div>

<div class="space-y-4">
  <!-- adds vertical spacing between direct
children -->
  <p>First</p>
  <p>Second</p>
</div>
```

The numbers (`1` , `2` , `4` , `8` ...) roughly map to a 0.25rem (4px) scale, so `p-4` is 1rem (16px), `p-8` is 2rem (32px). You'll memorize the common ones (`2` , `4` , `6` , `8`) quickly.

---

5. Colors

Every color has a name and a shade from 50 (lightest) to 950 (darkest):

```
<div class="bg-red-100 text-red-800">Light red
background, dark red text</div>
<div class="bg-gray-900 text-gray-50">Dark mode
style card</div>
<div class="border-2 border-green-500">Green
border</div>
```

Common color names: `slate`, `gray`, `red`, `orange`, `yellow`, `green`, `blue`, `indigo`, `purple`, `pink`. Picking `text-{color}-800` on `bg-{color}-100` is a common pattern for readable, low-contrast-risk color pairings.

---

6. Typography

```
<h1 class="text-3xl font-bold">Big Bold
Heading</h1>
<p class="text-sm text-gray-600">Small gray
paragraph</p>
<p class="italic underline">Italic and
underlined</p>
<p class="text-center leading-relaxed">Centered
text with relaxed line height</p>
```

Size scale: `text-xs`, `text-sm`, `text-base`, `text-lg`, `text-xl`, `text-2xl`, up through `text-9xl`.
Weight scale: `font-thin`, `font-normal`, `font-medium`, `font-semibold`, `font-bold`, `font-black`.

---

7. Layout: Flexbox and Grid Utilities

Tailwind's layout classes map almost directly to the CSS you already know:

```
<!-- Flexbox -->
<div class="flex items-center justify-between
gap-4">
  <span>Left</span>
  <span>Right</span>
</div>

<!-- Grid -->
<div class="grid grid-cols-3 gap-4">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
```

Tailwind class	Equivalent CSS
`flex`	`display: flex;`
`items-center`	`align-items: center;`
`justify-between`	`justify-content: space-between;`
`flex-col`	`flex-direction: column;`
`grid grid-cols-3`	`display: grid; grid-template-columns: repeat(3, 1fr);`

Tailwind class	Equivalent CSS
`gap-4`	`gap: 1rem;`

If you already know Flexbox and Grid, this section is mostly a vocabulary lookup, not new layout knowledge.

---

8. Sizing and Borders

```
<div class="w-64 h-32">Fixed width and
height</div>
<div class="w-full">Full width of parent</div>
<div class="max-w-md mx-auto">Constrained width,
centered</div>

<div class="border border-gray-300 rounded-
lg">Bordered box</div>
<div class="rounded-full">Fully rounded (good
for avatars)</div>
```

`w-full`, `w-1/2`, `w-1/3` use fractions of the parent; `w-64` uses the spacing scale; `max-w-md` caps width without forcing it.

---

9. States: Hover, Focus, and More

Instead of writing `:hover` in a separate CSS rule, prefix the utility with the state:

```
<button class="bg-blue-500 hover:bg-blue-700
text-white px-4 py-2 rounded">
  Click Me
</button>

<input class="border border-gray-300
focus:border-blue-500 focus:outline-none">
```

Any utility class can be prefixed this way: `hover:`, `focus:`, `active:`, `disabled:`. The base class sets the default look; the prefixed class only applies during that state.

---

10. Responsive Design

Tailwind uses breakpoint prefixes instead of separate `@media` blocks, applied mobile-first — the unprefixed class is the default (mobile), and each prefix applies from that screen width upward:

```
<div class="text-base md:text-lg lg:text-2xl">
  Small on mobile, larger on tablet, largest on
desktop
</div>

<div class="flex flex-col md:flex-row">
  Stacked on mobile, side-by-side from tablet
width up
</div>
```

Prefix	Applies from
(none)	all screen sizes
`sm:`	640px and up
`md:`	768px and up
`lg:`	1024px and up
`xl:`	1280px and up

---

11. Practice Project

Build a simple profile card using only utility classes:

```
<div class="max-w-sm mx-auto bg-white rounded-xl
shadow-lg p-6 flex flex-col items-center gap-3">
  <img src="avatar.jpg" class="w-20 h-20
rounded-full">
  <h2 class="text-xl font-semibold">Jane
Doe</h2>
  <p class="text-sm text-gray-500 text-
center">Product designer who loves clean
interfaces.</p>
  <button class="bg-blue-500 hover:bg-blue-700
text-white px-4 py-2 rounded-full">
    Follow
  </button>
</div>
```

Try modifying it: change the color palette, make it responsive with `md:flex-row` and a side-by-side layout, or

add a `dark:` prefix variant (Tailwind supports a `dark:` state prefix out of the box).

---

12. Where to Go Next

- **The `@apply` directive** — pull a repeated group of utilities into one custom class when a pattern repeats often
- `tailwind.config.js` — customize the color palette, spacing scale, and fonts to match a design system
- **Component libraries built on Tailwind** (like shadcn/ui) once you're comfortable with the utility approach
- **The official docs' search** (tailwindcss.com) — because there are hundreds of utility classes, looking things up as needed is normal, not a sign you don't know Tailwind

---

Tip: keep the official Tailwind docs open in a tab while practicing — most classes are guessable once you know the pattern, but the exact scale numbers and shade names take a little time to memorize.